

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Тверской государственный университет»

Факультет прикладной математики и кибернетики

Направление 02.03.02 Фундаментальная информатика и информационные
технологии

Профиль «Инженерия программного обеспечения»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА (ВЫПУСКНАЯ РАБОТА БАКАЛАВРА)

тема: «Обнаружение аномалий в сетевом
трафике»

Автор:

Иванова Екатерина
Валерьевна

Подпись _____

ДОПУЩЕН К ЗАЩИТЕ

Руководитель ООП

С. М. Дудаков

«_____» _____ 20____ г.

Научный руководитель:

Новикова Виктория
Николаевна, к. ф.-м. н.

Подпись _____

Abstract

Detecting anomalies in network traffic

by

Ivanova Ekaterina Valerevna

This project is about building a system that spots weird things happening in network traffic. It's a mix of different approaches, like the "bad packets" method, statistical methods, SVM, isolation forest, KMeans, and machine learning. Using fuzzy logic for these methods makes the analysis easier and less complicated. It also helps us figure out which approach should be used next or maybe one of them by itself, or all of them together. The built system is meant to automatically look at network activity and spot unusual or suspicious traffic. This could mean cyberattacks, someone trying to get in without permission, or other security risks. We spent a lot of time gathering and preparing the data necessary for writing the program. We wrote the code using Python and C++. For Python, we used some interesting libraries like Streamlit, pandas, NumPy, scikit-learn, and Matplotlib, among others. First, we studied how people usually try to find network anomalies. This included things like systems that look for known patterns (signatures), statistical ways of doing it, and machine learning. We checked out what's good and bad about each of these, especially when it comes to finding new threats nobody's seen before, and making sure the system works well even when network traffic is always changing. After looking at all that, we came up with what a good, adaptable anomaly detection system would need. Next, we explained how the software system we built is put together, mainly using Python. We really focused on the part that pulls out details about network traffic, like where data is coming from and going (IP addresses), the network ports, the protocols used, how many packets there are, and how much data is being sent. Our system has several different anomaly detectors working on their own. These use things like statistical analysis, isolation forest algorithms, SVM, KMeans clustering, and fuzzy logic rules. To make sure it spots things more accurately, we added a hybrid decision-making part that brings together what each of these detectors finds. The program also covers the user interface, which we built with Streamlit. This lets people load network data, start the analysis, and see the results visually. It can handle network traffic files in CSV and PCAP formats, and also offers ways to show detected anomalies and other

statistics in charts or graphs. We ran experiments to see how well our methods worked and how accurately they found anomalies in different network situations. The software system we created is a flexible platform for looking at network traffic. It brings together several ways to spot anomalies all in one place. This could be a good starting point for building even smarter systems to keep an eye on network security in the future.

Оглавление

Введение	6
Глава 1. Теория и основные определения	10
§ 1.1. Понятие сетевого трафика	10
§ 1.2. Описание потока по признакам	12
§ 1.3. Аномалии и их виды	12
§ 1.4. Подходы к обнаружению аномалий	13
§ 1.5. Статистический анализ как основа оценки отклонений	13
§ 1.6. Алгоритмы машинного обучения	14
§ 1.6.1. Метод k -средних	15
§ 1.6.2. Изоляционный лес	16
§ 1.6.3. Метод опорных векторов	17
§ 1.6.4. Случайный лес	19
§ 1.7. Нечеткая логика	19
§ 1.7.1. Основные моменты	20
§ 1.7.2. Этапы нечеткого вывода	21
§ 1.7.3. Адаптация для задачи агрегирования детекторов	21
§ 1.8. Сравнение методов	22
Глава 2. Описание программного обеспечения	24
§ 2.1. Требования, предъявляемые к программному обеспечению	24
§ 2.2. Выбор средств разработки	25
§ 2.3. Архитектура программы	26
§ 2.4. Диаграмма архитектуры и поток данных	26
§ 2.5. Извлечение и подготовка данных	28
§ 2.5.1. Инженерные особенности парсинга файлов PCAP	28
§ 2.6. Реализованные детекторы	29
§ 2.6.1. Детектор k -средних	30
§ 2.6.2. Лес изоляции	31
§ 2.6.3. SVM	31

§ 2.6.4. Статистический детектор	32
§ 2.6.5. Правилковой детектор	32
§ 2.6.6. Модель машинного обучения	32
§ 2.6.7. Сравнительный анализ работы детекторов на тестовых данных	33
§ 2.6.8. Анализ производительности системы	35
§ 2.7. Модуль нечеткой логики	36
§ 2.8. Пользовательский интерфейс	37
§ 2.9. Преимущества разработанной системы	38
Глава 3. Программная реализация	39
Заключение	44
Список литературы	45
Приложение А. Код программы	47

Введение

Актуальность

В условиях распространения технологического прогресса объем сетевого взаимодействия непрерывно растет, вместе с ним расширяется диапазон угроз информационной безопасности. Современные компьютерные сети используются для клиентских запросов, передачи рабочих данных, телеметрии, файлового обмена, удаленного доступа и множества других сценариев. Такое разнообразие делает сетевой трафик одним из наиболее информативных источников сведений о состоянии информационной системы.

Традиционные средства защиты информации, основанные на сигнатурных правилах, эффективно выявляют известные типы атак, но их возможности ограничены в ситуациях, когда характер угрозы изменяется или же атакующий использует не фиксируемые ранее шаблоны поведения. В таких условиях растет значимость методов обнаружения аномалий, ориентированных не на конкретную сигнатуру, а на выявление отклонений от нормального поведения системы.

Особый интерес представляют гибридные системы, которые объединяют в себе несколько механизмов анализа. Они могут сочетать статистические методы, методы машинного обучения, экспертные правила и другие алгоритмы. Подобное сочетание методов делает систему наиболее устойчивой и надежной к уязвимостям, так как частные недостатки одного метода уравниваются преимуществами другого.

Обзор литературы

При исследовании темы были рассмотрены основные подходы к анализу сетевого трафика [5; 10], методы классификации [8] и выявления нарушений [1; 11; 12], а также основы нечеткой логики [9]. Теоретической базой послу-

жили книги и статьи по статистическому анализу, кластеризации, методам обнаружения аномалий с учителем и без учителя [2—4; 7], а также литературные источники, которые посвящены экспертным системам и гибридным архитектурам. В ходе изучения литературных источников был установлен такой факт, что практические системы обнаружения аномалий, в большинстве случаев, используют не один метод, а набор комплементарных механизмов [3; 11]. Это связано с тем, что поведение сетевого трафика отличается высоким разнообразием [10]. Аномальность может проявляться через большие объемы данных, частые короткие соединения, нетипичные порты или нестандартную последовательность действий [6; 7; 11].

Цель и задачи

Цель работы. Проанализировать сетевой трафик и выявить отклонения от норм поведения, которые могут указывать на вторжения или вредоносную активность.

Задачи

- 1) Сбор и подготовка датасета сетевого трафика с метками.
- 2) Проектирование пайплайн извлечения признаков (flow/packet-level).
- 3) Исследование и реализация нескольких методов обнаружения отклонений.
- 4) Оценка эффективности работы алгоритмов по набору метрик.
- 5) Разработка интерфейса приложения для анализа трафика.

Методы и средства

В работе применялись методы статистического анализа, кластеризации, обнаружения аномалий без учителя, обучения с учителем и нечеткой логики. Для программной реализации были выбраны языки программирования C++ и Python. Такой выбор был сделан неслучайно: C++ обеспечивает высокую эффективность, а Python — скорость выполнения, удобство написания кода, использования библиотек существующих методов и машинного обучения, удобство подключения веб-интерфейса и т. д. В нашем случае использовались

следующие библиотеки Python: streamlit используется для реализации веб-интерфейса; pandas — для удобной загрузки, обработки табличных данных, анализа и структурирования данных; numpy — для работы с многомерными массивами; scikit-learn — для обучения модели в машинном обучении; matplotlib — для создания графиков и диаграмм; и другие. Разработка производилась в среде разработки Clion в операционной системе Linux. При выполнении программа открывает веб-интерфейс на локальном хосте.

Основные результаты

- 1) Выполнен сбор и подготовка датасета сетевого трафика с метками, а именно: проведена работа с исходными наборами данных, которая включает в себя очистку от дублей, обработку пропущенных значений и нормализацию числовых параметров; выполнено кодирование четких признаков и балансировка классов, что обеспечило качественную основу для обучения моделей обнаружения аномалий.
- 2) Спроектирован пайплайн извлечения признаков на уровне flow, что включало в себя разработанную систему агрегации пакетов в потоки с извлечением статистических характеристик (длительности соединения, количества пакетов и байтов в обоих направлениях, временных интервалов) и реализованные механизмы вычисления производных признаков, которые характеризуют поведение сетевого трафика.
- 3) Исследованы и реализованы несколько методов обнаружения отклонений, в том числе применены алгоритмы как с учителем (Random Forest), так и без учителя (Isolation Forest, One-Class SVM); разработана система агрегирования оценок от независимых детекторов, где каждый формирует свою оценку аномальности, а итоговое решение принимается на основе всех оценок.
- 4) Проведена оценка эффективности работы алгоритмов по набору метрик. Сравнительный анализ показал, что гибридный подход превосходит отдельные методы по устойчивости к различным типам аномалий и снижает количество ложных срабатываний у всех методов.
- 5) Создан интерфейс, позволяющий визуализировать результаты анализа в удобной форме, в частности реализована возможность обработки загружаемых файлов форматов PCAP и CSV, интерактивное отображение

статистики по потокам, визуализация аномальности пакетов при помощи графиков и диаграмм.

Ко всему прочему, в созданном интерфейсе добавлена возможность менять порог аномальности пакетов и скачивание файла CSV с информацией о полученных аномальных потоках. Также по ходу работы над проектом была проведена диагностика системы, добавлены тесты для проверки работоспособности частей программы (проверка правильности работы детекторов, экстрактора потоков и т. д.).

Структура работы

Работа состоит из введения, трех основных глав, заключения и приложения. В первой главе рассматриваются теоретические основы анализа сетевого трафика, понятие аномалии, а также методы обнаружения отклонений, включая статистический анализ, алгоритмы машинного обучения и нечеткая логика. Во второй главе описываются архитектура программной системы, используемые алгоритмы, механизм объединения результатов, а также особенности практической реализации и пользовательского интерфейса. В третьей главе показывается работа созданной программы и ее основные нюансы. Также в работу включено приложение А. В этом приложении можно ознакомиться с архитектурой проекта и кодом программ, реализующих описанные далее алгоритмы.

Глава 1

Теория и основные определения

В этой главе представлена теоретическая основа, которая пригодилась в написании программы. Глава посвящена анализу проблемы поведенческого контроля сетевых взаимодействий. Здесь мы раскроем понятия сетевого трафика и потока, приведем признаковое описание сессий и обоснуем необходимость нормализации метрик. Также разберем классификацию по типам аномалий (точечные, контекстные, коллективные) с привязкой к реальным сценариям сетевой безопасности, проведем сравнительный разбор подходов: от статистических фильтров и алгоритмов обучения без учителя до гибридных моделей с учителем и нечеткой логики. На основе анализа ограничений каждого метода сформируем теоретическое обоснование гибридной архитектуры, где отдельные оценки компенсируют недостатки отдельных детекторов.

§ 1.1. Понятие сетевого трафика

Определение 1 (Сетевой трафик). Сетевой трафик — это процесс обмена информацией между устройствами, подключенными к компьютерной сети. В ходе такого взаимодействия данные передаются в виде отдельных пакетов, которые перемещаются от отправителя к получателю через сетевую инфраструктуру. Источниками трафика могут выступать пользовательские приложения, веб-сервисы, операционные системы, облачные платформы и различные сетевые устройства [5; 10].

Каждый передаваемый пакет содержит в себе служебную информацию и пользовательские данные [10]. Служебная часть включает заголовки сете-

вых протоколов, в которых указываются адреса отправителя и получателя, используемый протокол передачи, номера портов, информация о маршрутизации и другие параметры, которые необходимы для корректной доставки данных. Полезная нагрузка пакета содержит передаваемую информацию, например текст, мультимедийные данные или команды приложений. Анализ сетевого трафика позволяет получить представление о характере работы сети, активности пользователей и функционировании программных систем. По параметрам пакетов можно выявлять закономерности взаимодействия узлов, определять интенсивность обмена данными, фиксировать перегрузки сети, а еще обнаруживать подозрительную или аномальную активность. В современных системах информационной безопасности исследование сетевого трафика является одним из ключевых способов мониторинга состояния сети и выявления потенциальных угроз.

На практике анализ отдельных пакетов зачастую оказывается не достаточно эффективным из-за большого объема передаваемых данных. По этой причине трафик обычно преобразуют в форму потока.

Определение 2 (Поток). Поток представляет собой логически связанную последовательность пакетов, объединенных по каким-то общим характеристикам. Чаще всего для формирования потока используются IP-адрес источника и назначения, номера портов, тип транспортного протокола и направление передачи данных [1; 5].

После формирования потоков вычисляются их агрегированные характеристики. К таким параметрам относятся количество переданных пакетов, общий объем данных в байтах, продолжительность соединения, средний размер пакета, интенсивность передачи информации и другие показатели. Использование потокового представления позволяет существенно уменьшить объем анализируемых данных и одновременно сохранить наиболее значимые сведения о сетевом взаимодействии. Поточковый анализ широко применяется в системах мониторинга и обнаружения сетевых атак. На основе статистических характеристик потоков можно выявлять нетипичное поведение, например резкое увеличение объема передаваемых данных, необычную частоту соединений или нестандартные схемы взаимодействия между узлами сети. Это делает потоковую модель удобной основой для построения систем обнаружения аномалий и анализа сетевой безопасности.

§ 1.2. Описание потока по признакам

Поток можно представить как объект анализа в виде вектора признаков $x = (x_1, x_2, \dots, x_n)$, где x_i — отдельный признак, который описывает сетевое взаимодействие [8]. В рассматриваемой задаче большое значение имеют как числовые признаки, так и факторные параметры.

Перед подачей на модель признаки обычно стандартизируются. Если признак имеет среднее значение μ и стандартное отклонение σ , то стандартизованное значение вычисляется как $z = \frac{x-\mu}{\sigma}$ [3; 8]. Такое преобразование позволяет привести признаки к базовому масштабу.

§ 1.3. Аномалии и их виды

Определение 3 (Аномалия). Аномалия в сетевом трафике — это наблюдение или последовательность наблюдений, которое заметно отличается от типичного поведения системы [3; 11]. Аномалия не всегда означает атаку: в ряде случаев отклонение может быть связано с обновлением сервисов, изменением нагрузки или техническими работами.

Аномалии можно классифицировать на следующие виды.

- 1) Точечные аномалии — отдельные экземпляры данных, существенно отличающиеся от остальной части набора данных и являющиеся аномальными по отношению к остальным данным. Это единоразовые выбросы, резко выделяющиеся на фоне остальных наблюдений.
- 2) Контекстные аномалии — наблюдения, которые являются аномальными только при заданном контексте. Аномальность определяется формально через отклонение поведенческих признаков от типичного поведения при заданных условиях контекста (время, место, режим, категория и т. д.).
- 3) Коллективные аномалии — группы взаимосвязанных наблюдений, в совокупности они образуют нетипичный паттерн, хотя отдельные точки данных могут быть нормальными. В отличие от точечных аномалий, коллективные аномалии проявляются на уровне групп или последовательностей данных, при этом демонстрируя необычное поведение или характеристики, которые значительно расходятся с коллективной нормой [3; 4].

Для сетевого трафика типичными примерами аномального поведения являются массовое сканирование портов, многократные попытки аутентификации, резкий рост числа коротких соединений, передача чрезмерно больших объемов данных и использование нетипичных протоколов.

§ 1.4. Подходы к обнаружению аномалий

В сфере обнаружения аномалий есть несколько рабочих подходов — у каждого свои плюсы и ограничения.

- 1) Сигнатурный — самый прямолинейный: он ищет совпадения с уже известными шаблонами атак. По сути, это как база сигнатур вирусов в антивирусе: если что-то совпадает — срабатывает тревога.
- 2) Статистический исходит из того, что у «нормального» поведения есть устойчивые характеристики — например, признаки укладываются в какое-то распределение. Отклонения от этой нормы и считаются аномалиями.
- 3) Подходы на основе машинного обучения идут дальше: они не опираются на жесткие правила, а учатся на данных. Такие модели способны находить не очевидные, сложные связи между признаками, которые человеку заметить трудно.
- 4) Гибридный подход берет лучшее от разных методов. Комбинируя их, можно посмотреть на данные с нескольких сторон, и в конечном итоге получить более надежное решение [3; 11].

§ 1.5. Статистический анализ как основа оценки отклонений

Один из наиболее простых способов выявления аномалии — вычисление z -оценки: $z = \frac{x - \mu}{\sigma}$. Если модуль z -оценки велик, значение признака существенно отличается от среднего по выборке. Такой подход особенно полезен для быстрого выявления грубых отклонений.

Преимущество статистического метода состоит в его простоте и интерпретируемости.

Недостаток состоит в том, что он слабо учитывает сложные нелинейные зависимости между признаками, что делает его слабоэффективным.

§ 1.6. Алгоритмы машинного обучения

Методы машинного обучения играют чуть ли не основную роль в современных системах обнаружения аномалий, так как они позволяют выявлять скрытые закономерности в данных и адаптироваться к изменяющемуся характеру сетевого трафика.

При выборе алгоритмов для системы обнаружения аномалий учитывались следующие положения: устойчивость к различным шумам, возможность работы с ограниченным количеством размеченных данных, вычислительная эффективность, интерпретируемость результатов и применимость к потоковым сетевым данным.

В рамках работы были рассмотрены как методы обучения без учителя, так и с учителем. Методы без учителя особенно важны для задач сетевой безопасности, так как в реальных условиях получение большого количества корректно размеченных данных затруднено.

Таблица 1.1. Сравнение подходов к обнаружению аномалий.

Подход	Преимущества	Недостатки
Без учителя	обнаружение неизвестных атак	высокий риск ложных срабатываний
С учителем	высокая точность	зависимость от обучающей выборки
Гибридный	устойчивость и адаптивность	очень высокая сложность реализации

Исходя из данных в таблице 1.1 можем заметить, что гибридный подход объединяет в себе сильные стороны обоих направлений: он приспосабливается к новым атакам, снижая при этом долю ошибок за счет взаимной проверки детекторов. Несмотря на выросшую сложность проектирования, именно такая архитектура обеспечивает более устойчивую работу в условиях изменчивого сетевого трафика, что и обосновывает ее более детальное рассмотрение в рамках данной работы.

§ 1.6.1. Метод k -средних

Алгоритм k -средних (KMeans) относится к методам кластеризации. Его идея заключается в том, чтобы разделить объекты на k кластеров таким образом, чтобы расстояние внутри кластеров было минимальным [8].

Целевая функция, которая нужна для количественной оценки качества кластеризации, имеет вид:

$$J = \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2,$$

где

- J — значение целевой функции (суммарное внутрикластерное отклонение);
- k — количество кластеров;
- S_i — i -й кластер (множество точек, отнесенных к этому кластеру);
- x — отдельная точка данных (вектор признаков) из кластера S_i ;
- μ_i — центроид (центр) i -го кластера;
- $\|x - \mu_i\|^2$ — квадрат евклидова расстояния между точкой x и центроидом μ_i .

В контексте обнаружения аномалий объект, который находится далеко от центра кластера, может рассматриваться как подозрительный.

Преимущества:

- 1) легко понять и реализовать, есть готовые библиотеки (к примеру, scikit-learn);
- 2) линейная сложность относительно объема данных, подходит для больших наборов; низкая скорость обработки данных;
- 3) быстрый результат на «чистых» данных с явными кластерами;
- 4) центроиды — средние значения признаков, их легко анализировать и проверять;
- 5) широкая применимость — группировка клиентов, обработка изображений, финансовый анализ и т. д.;

- 6) каждый объект относится строго к одному кластеру, что говорит о четком разделении.

Недостатки:

- 1) необходимо заранее знать количество кластеров. Либо задать число k , либо подбирать вручную;
- 2) результат зависит от начальной точки;
- 3) алгоритм предпочитает одинаковые по размеру кластеры;
- 4) требует идеальной подготовки данных, т. е. нужно масштабировать признаки, убирать категориальные, так как на «сырых» данных метод работать не будет;
- 5) жесткое разделение — точка может принадлежать только одному кластеру;
- 6) боится выбросов — одна аномальная точка может сдвинуть центроид кластера;
- 7) на очень больших данных начинает «тормозить», если не использовать оптимизации.

§ 1.6.2. Изоляционный лес

Лес изоляции (изоляционный лес, Isolation Forest) основан на случайной изоляции объектов [2; 3]. Алгоритм строит ансамбль деревьев, в которых данные разделяются случайными порогами. Аномальные объекты, как правило, изолируются быстрее нормальных. Оценка аномальности связана со средней длиной пути в деревьях изоляции для каждой точки:

$$s(x, m) = 2^{-\frac{E(h(x))}{c(m)}},$$

где $E(h(x))$ — средняя длина пути для точки x по всем деревьям, а $c(m)$ — нормализационная константа (средняя длина пути в сбалансированном бинарном дереве размера m).

При этом результаты оцениваются следующим образом:

- $s \approx 1$ — точка с высокой вероятностью является аномалией (изолируется быстро, короткий путь в деревьях);

- $s < 0,5$ — точка скорее всего нормальная (требуется много разделений для изоляции, длинный путь);
- $s \approx 0,5$ — данные не содержат явных аномалий.

Преимущества метода:

- 1) работает быстрее традиционных методов из-за использования подвыборок данных;
- 2) не нужно хранить всю выборку, из чего следуют низкие требования к памяти;
- 3) хорошо работает с большими и многомерными наборами данных;
- 4) не требует обучения на «чистых» данных. Может работать с наборами, содержащими аномалии;
- 5) позволяет настраивать уровень детализации обнаружения без переобучения, что делает метод более гибким.

Недостатки:

- 1) когда аномалии расположены близко к нормальным точкам их сложнее изолировать;
- 2) алгоритм случайным образом выбирает признаки для разбиения, что может быть не очень оптимально для коррелированных данных;
- 3) нормальные точки на краях распределения могут получить высокие оценки аномалии, из-за чего могут быть ложные срабатывания;
- 4) результат зависит от размера подвыборки и количества деревьев, что говорит о чувствительности к гиперпараметрам.

§ 1.6.3. Метод опорных векторов

Метод опорных векторов, или иначе One-Class SVM, строит границу нормального поведения в пространстве признаков. Все точки (опорные векторы), находящиеся за пределами этой области, рассматриваются как потенциальные аномалии [8]. По итогу получается компактная модель, зависящая только от опорных векторов. Правило классификации через этот метод:

$$f(x) = \text{sign} \left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \right),$$

где $f(x)$ выдает $+$ или $-$, всего два класса.

Здесь:

- $\alpha_i > 0$ — коэффициент для обучения модели;
- $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ — трюк с заменой скалярного произведения;
- $\frac{1}{2}\|w\|^2 \rightarrow \min$ — задача минимизации, где главная цель — максимизировать зазор между классами (или минимизировать норму w , w = вектор весов);
- $\frac{1}{2}\|w\|^2 + C \sum_{i=1}^n \xi_i \rightarrow \min$ — мягкие границы SVM, где
- C — параметр штрафа за ошибки;
- ξ_i — переменные-допуски (учитывают выбросы);
- $y_i(w \cdot x_i + b) \geq 1$ — условие корректной классификации;
- $w \cdot x + b = 0$ — разделяющая гиперплоскость.

Преимущества метода опорных векторов:

- 1) работает эффективно, когда число признаков превышает число объектов;
- 2) универсальность — подходит для задач классификации и регрессии;
- 3) позволяет строить нелинейные границы принятия решений без явного преобразования признаков;
- 4) устойчивость к переобучению — оптимизирует зазор, что повышает обобщающую способность;
- 5) может показывать хорошие результаты даже на относительно малых объемах данных;
- 6) итоговая модель зависит только от не большого числа ключевых объектов (опорных векторов), что упрощает интерпретацию;
- 7) отсутствие локальных минимумов — задача оптимизации выпуклая, гарантирует нахождение глобального оптимума;
- 8) в отличие от некоторых других методов, не требует строгого исключения коррелирующих признаков.

Недостатки:

- 1) обучение имеет сложность порядка $O(n^2) - O(n^3)$, что затрудняет применение на очень больших наборах данных и влечет за собой высокие вычислительные затраты;
- 2) чувствительность к выбору ядра и гиперпараметров — требуется тщательная настройка (C , γ и др.), часто через кросс-валидацию;
- 3) разные масштабы признаков искажают расстояния, ухудшая качество модели, поэтому признаки нужно масштабировать;
- 4) сложно интерпретировать модель, особенно при использовании нелинейных ядер; сложно объяснить, почему сделан тот или иной прогноз;
- 5) SVM выдает знак расстояния до разделяющей гиперплоскости, а не прямое разделение на вероятностные классы; для вероятностей нужны дополнительные методы;
- 6) проблемы с зашумленными данными, так как выбросы могут стать опорными векторами и исказить границу решения;
- 7) стандартный SVM бинарен, многоклассовые задачи требуют стратегий типа «один против одного» или «один против всех»;
- 8) если признаков слишком много, а объектов мало, то риск переобучения возрастает, несмотря на регуляризацию.

§ 1.6.4. Случайный лес

Random Forest является ансамблевым методом, объединяющим множество деревьев решений. Итоговый прогноз формируется на основе усреднения или голосования отдельных деревьев:

$$f(x) = \frac{1}{T} \sum_{t=1}^T h_t(x).$$

Метод устойчив к шуму, хорошо работает на табличных данных и позволяет получать интерпретируемые оценки важности признаков.

§ 1.7. Нечеткая логика

Нечеткая логика используется в ситуациях, когда границы между понятиями размыты [9]. В отличие от классической логики, где высказывание либо

истинно, либо ложно, нечеткая логика допускает промежуточные значения истинности от 0 до 1.

Для нечеткого множества степень принадлежности элемента x задается функцией $\mu(x) \in [0, 1]$. Это позволяет описывать такие термины, как «низкий риск», «средний риск» и «высокий риск», не ограничиваясь бинарной классификацией.

В классической системе нечеткого вывода выделяют три этапа: фаззификация, нечеткий вывод и дефаззификация. В рассматриваемой работе используется упрощенная форма нечеткой логики, при которой числовые оценки сначала относятся к лингвистическим категориям, а после сворачиваются в единую итоговую величину [9].

В задачах обнаружения аномалий в сетевом трафике часто возникает необходимость объединения нескольких разнородных показателей в единую итоговую оценку. Традиционные методы суммирования или голосования обладают рядом ограничений: они предполагают линейную зависимость между частными оценками и не учитывают неопределенность, присущую каждому детектору в отдельности. Нечеткая логика позволяет преодолеть эти ограничения за счет работы с лингвистическими переменными и правилами вывода.

§ 1.7.1. Основные моменты

В классической теории нечетких множеств степень принадлежности элемента x к некоторому множеству A задается функцией принадлежности

$$\mu_A(x) \in [0, 1].$$

В отличие от бинарной логики, где объект принадлежит множеству или не принадлежит, нечеткий подход допускает промежуточные состояния. Это особенно важно при интерпретации вероятностных выходов алгоритмов машинного обучения.

Для описания уровня аномальности обычно вводятся термы: LOW (низкий риск), MEDIUM (средний риск), HIGH (высокий риск). Каждый терм формализуется через функцию принадлежности. Наиболее распространенными формами являются триангулярные и трапециевидные функции. Например,

триангулярная функция для терма MEDIUM может быть записана так:

$$\mu_{\text{MEDIUM}}(x) = \begin{cases} 0, & x \leq a, \\ \frac{x-a}{b-a}, & a < x \leq b, \\ \frac{c-x}{c-b}, & b < x < c, \\ 0, & x \geq c, \end{cases}$$

где a, b, c — параметры, определяющие положение и ширину функции на числовой оси.

§ 1.7.2. Этапы нечеткого вывода

Классическая система нечеткого вывода Мамдани включает три этапа:

Фаззификация — перевод четких числовых оценок детекторов в степени принадлежности лингвистическим термам.

Нечеткий вывод — применение базы правил вида ЕСЛИ <условие> ТО <заключение>, где условия и заключения выражены через термы. Активация правил обычно выполняется с помощью операций $\min$ (И) или prod .

Дефаззификация — преобразование результирующего нечеткого множества обратно в четкое числовое значение. Наиболее распространенный метод — центр тяжести (Центроид):

$$x^* = \frac{\int x \mu(x) dx}{\int \mu(x) dx}.$$

§ 1.7.3. Адаптация для задачи агрегирования детекторов

В разрабатываемой системе применяется упрощенная схема, ориентированная на высокую скорость работы и интерпретируемость. Вместо непрерывных функций принадлежности используются пороговые интервалы:

LOW: $x \in [0, 0.3)$, MEDIUM: $x \in [0.3, 0.7)$, HIGH: $x \in [0.7, 1.0]$.

База правил формируется на основе экспертных представлений о совместном поведении детекторов. Например, если большинство детекторов выдает HIGH, итоговая оценка максимизируется; если преобладает LOW, поток считается нормальным; при смешанных сигналах применяется промежуточная оценка. Такой подход сохраняет гибкость нечеткого вывода, но исключает

вычислительные накладные расходы, связанные с интегрированием и тонкой настройкой функций принадлежности. Это оправдано в исследовательской системе, где приоритет отдается прозрачности логики принятия решений и возможности быстрого проектирования и прототипирования.

§ 1.8. Сравнение методов

Для удобства оценки всех методов, сравнения их друг с другом, представим все методы в виде таблицы 1.2.

Таблица 1.2. Сравнение методов.

Метод	Тип	Достоинства	Ограничения
Плохих пакетов	на основе правил, ручную ввод плохих пакетов	простота реализации, высокая скорость, эффективен для известных угроз	не обнаруживает новые атаки, требует обновления списка плохих портов, ложные срабатывания
z -оценка	статистический	простота, высокая скорость	слабая работа со сложными зависимостями
k -средних	без учителя	наглядность, кластерная структура	чувствительность к числу кластеров
Лес изоляции	без учителя	эффективен для выбросов	требует настройки параметров
Метод опорных векторов (SVM)	без учителя	гибкая граница нормы	вычислительная сложность
Случайный лес	с учителем	точность, устойчивость	требует размеченных данных
Нечеткая логика	агрегирование	гибкость, понятность	зависит от качества правил

Из сравнения методов мы снова можем увидеть, что универсального решения не существует. Простые подходы, вроде фильтрации плохих пакетов и z -оценки, работают быстро и легко внедряются, но бесполезны против сложных и неизвестных атак. Методы машинного обучения без учителя (k -средних,

изоляционный лес, SVM) способны находить аномалии без предварительной разметки, но каждый из них имеет свои минусы: кто-то требует тонкой настройки параметров, кто-то — больших вычислительных ресурсов. Случайный лес показывает высокую точность, но ему нужны размеченные данные, которые не всегда есть под рукой. Нечеткая логика сама по себе не обнаруживает аномалии, но служит удобным инструментом для объединения результатов нескольких детекторов. Именно поэтому в работе выбран гибридный подход с агрегацией из нечеткой логики: комбинация методов позволяет компенсировать недостатки одних алгоритмов преимуществами других.

Глава 2

Описание программного обеспечения

Разработанная программная система предназначена для анализа сетевого трафика, представленного в формате CSV или PCAP(а также PCANG), и выявления аномальных потоков. Основная идея решения состоит в том, чтобы не ограничиваться единственным алгоритмом анализа, а использовать несколько взаимодополняющих детекторов.

§ 2.1. Требования, предъявляемые к программному обеспечению

В ходе анализа предметной области и постановки задачи были сформулированы следующие требования к разрабатываемой системе:

- поддержка загрузки и парсинга сетевых данных в форматах CSV и PCAP с корректной обработкой поврежденных или неполных пакетов;
- автоматическое агрегирование пакетов в логические потоки (flow) по кортежу из 5 элементов (5-кортеж) и вычисление статистических признаков (объем данных, длительность сессии, интенсивность, средний размер пакета и др.);
- реализация гибридного анализа с использованием нескольких независимых детекторов: статистического, кластерного (метод k -средних), изоляционного (Isolation Forest), метода опорных векторов (One-Class SVM), ансамблевого (случайный лес) и правилowego;

- применение упрощенной нечеткой логики для агрегации частных оценок аномальности и формирования итогового показателя подозрительности; порог срабатывания настраивается;
- наличие веб-интерфейса для загрузки файлов, визуализации распределения оценок детекторов, просмотра таблицы аномальных потоков и экспорта результатов в формате CSV;
- модульная архитектура с единым интерфейсом детекторов, которая позволяет добавлять новые алгоритмы анализа без изменения основного пайплайна обработки данных.

§ 2.2. Выбор средств разработки

Задача обнаружения аномалий в сетевом трафике требует эффективной работы с табличными данными, реализации алгоритмов машинного обучения, быстрого прототипирования пайплайнов обработки и создания удобного интерфейса для аналитика. Учитывая эти требования, а также необходимость поддержки расширяемой модульной архитектуры, в качестве основного языка разработки был выбран Python 3.10. Такой выбор сделан неслучайно и он обусловлен значимыми факторами.

- **Зрелая экосистема для сложной библиотеки ML.** Библиотеки `pandas` и `numpy` обеспечивают высоко оптимизированную обработку многомерных массивов и табличных признаков, а `scikit-learn` предоставляет готовые, хорошо протестированные реализации всех требуемых алгоритмов, что существенно ускоряет экспериментальную часть.
- **Удобство работы с сетевыми данными.** Библиотека `scapy` позволяет гибко парсить бинарные PCAP файлы, извлекать заголовки сетевых и транспортных протоколов и реализовывать потоковую обработку без полной загрузки лога в оперативную память.
- **Быстрое создание пользовательского интерфейса.** Фреймворк `Streamlit` позволяет развернуть интерактивное веб-приложение на чистом Python, при этом явно минимизируя затраты на разработку фроненда и серверной части, а также сохраняя местную (нативную) интеграцию с аналитическим ядром.

- **Среда разработки и контроль версий.** Разработка велась в Clion с использованием виртуальных окружений (`venv`), что соответствует современным стандартам написания исследовательских и инженерных проектов.

При том, что большая часть работы была сделана при помощи Python, запуск производится с помощью языка C++. Между C++ и Python проведен мост для корректной работы программы. Python здесь используется в качестве удобства написания кода, высокой скорости, а C++ для ресурсоемкости.

§ 2.3. Архитектура программы

Архитектура разработанной системы построена как многоуровневая модульная структура, в которой отдельные компоненты отвечают за конкретные этапы обработки сетевого трафика. Такой подход обеспечивает масштабируемость, повторное использование компонентов и упрощает сопровождение программного комплекса.

На концептуальном уровне архитектуру можно разделить на следующие слои: слой загрузки и предварительной обработки данных, слой извлечения потоков и признаков, слой аналитической обработки, слой нечеткой агрегации и слой пользовательского интерфейса.

Каждый слой взаимодействует с соседними через стандартизированные структуры данных, представленные в виде таблиц признаков. Ключевым преимуществом подобной архитектуры является слабая связанность компонентов.

Система реализует следующий pipeline обработки: загрузка файла → парсинг пакетов → извлечение потоков → вычисление признаков → нормализация → анализ детекторами → нечеткая агрегация → формирование результата → визуализация.

§ 2.4. Диаграмма архитектуры и поток данных

Логическая схема взаимодействия компонентов включает несколько последовательных этапов: пользователь загружает CSV или PCAP файл; модуль парсинга определяет тип файла и извлекает данные; пакеты агрегируются в потоки; для каждого потока формируется вектор признаков; числовые

признаки нормализуются; набор потоков передается в детекторы; каждый детектор формирует собственный anomaly score; модуль нечеткой агрегации объединяет частные оценки; итоговые результаты отображаются в интерфейсе и передаются через веб-интерфейс (см. рис. 2.1).

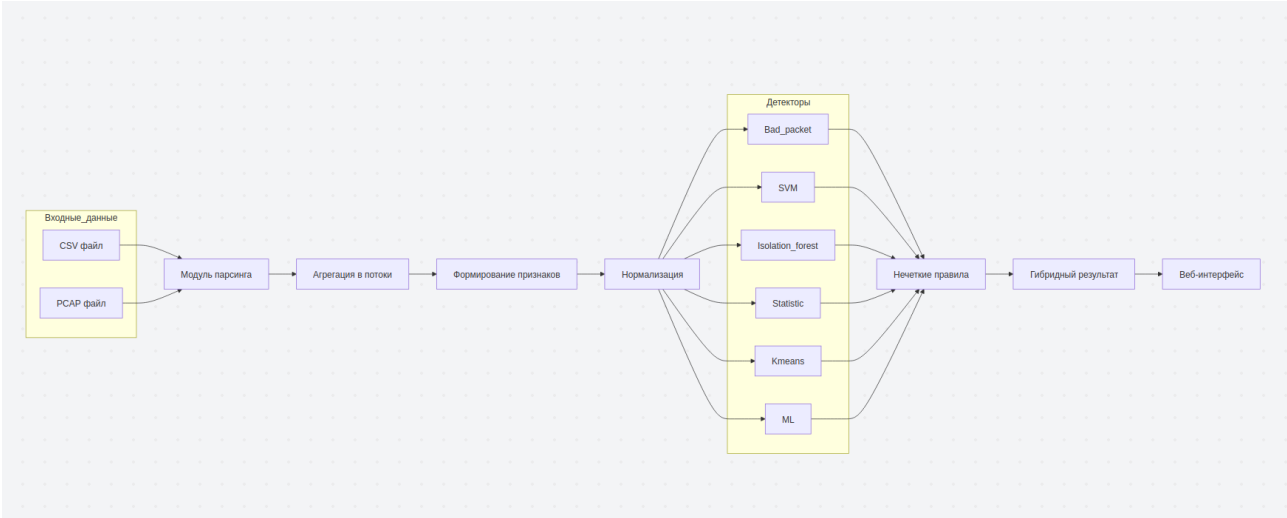


Рис. 2.1. Диаграмма архитектуры.

С помощью таблицы 2.1 опишем то, из каких составляющих у нас агрегированные пакеты:

Таблица 2.1. Признаки сетевого трафика и методы их обработки.

Признак	Тип данных	Описание	Метод обработки
src_ip	категориальный	IP-адрес источника	используется как идентификатор
dst_ip	категориальный	IP-адрес назначения	используется как идентификатор
src_port	числовой	порт источника	стандартизация
dst_port	числовой	порт назначения	стандартизация
protocol	категориальный	сетевой протокол	кодирование
bytes	числовой	объем переданных данных	нормализация
packets	числовой	число пакетов	нормализация
bytes/packets	производный	средний размер пакета	вычисляемый признак

Из таблицы видно, что для анализа сетевого трафика используется комбинация разнохарактерных признаков. IP-адреса (`src_ip`, `dst_ip`) и протокол (`protocol`) носят категориальный характер и служат для идентификации участников взаимодействия, поэтому не подвергаются масштабированию. Числовые метрики — порты, объемы данных и количество пакетов — требуют нормализации, так как имеют разные диапазоны значений и без приведения к единому масштабу могли бы исказить работу алгоритмов машинного обучения. Отдельно стоит производный признак `bytes/packets` (количество байт на пакет в среднем), который вычисляется на этапе предобработки и характеризует средний размер пакета в потоке. Эта метрика часто оказывается более информативной для выявления аномалий, таких как сканирование портов или утечка данных. Такой подход к обработке признаков позволяет сохранить информативность исходных данных и обеспечить корректную работу детекторов аномалий.

§ 2.5. Извлечение и подготовка данных

Подготовка данных является одним из наиболее важных этапов системы обнаружения аномалий, потому что качество признакового пространства напрямую влияет на эффективность алгоритмов машинного обучения.

При работе с CSV-файлами система проверяет наличие обязательных полей и корректность типов данных. Для PCAP файлов выполняется последовательный разбор пакетов сетевого уровня. Модуль извлечения потоков группирует пакеты по 5-кортежу (`src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`), после чего вычисляет агрегированные характеристики.

После формирования признаков выполняется их масштабирование. Для большинства алгоритмов используется описанная в главе 1 стандартизация. Стандартизация необходима, поскольку признаки имеют разные диапазоны значений.

§ 2.5.1. Инженерные особенности парсинга файлов PCAP

Парсинг файлов формата PCAP(и PCAPNG) является одним из наиболее важных этапов работы системы, так как именно на нем формируется первичная основа для дальнейшего извлечения признаков и запуска детекторов.

В реальных условиях сетевые дампы часто имеют объем в сотни мегабайт или даже несколько гигабайт, могут содержать битые пакеты, обрезанные ТСР-сессии или же быть захваченными разными инструментами с неоднородными заголовками. Загрузка всего файла в чистом виде в оперативную память или наивная обработка без контроля целостности данных приводит к аварийному завершению приложения (в самых плохих случаях), искажению агрегированных метрик и, как следствие, к росту количества ложных срабатываний на этапе анализа.

С целью обеспечения устойчивости, высокой скорости обработки и правильности признакового пространства в модуле парсинга были реализованы инженерные решения, которые направлены на уменьшение рисков и оптимизацию потребления вычислительных ресурсов. Основные особенности реализации включают:

- 1) обработку повреждённых пакетов;
- 2) фильтрацию неполных ТСР-сессий;
- 3) поддержку различных форматов захвата;
- 4) минимизацию потребления оперативной памяти;
- 5) потоковую обработку данных без полной загрузки лога в память.

Более того, в системе предусмотрен механизм проверки входных данных на этапе инициализации. Реализована проверка наличия обязательных заголовков, контроль цельной структуры файла, обработка пустых наборов пакетов и отсеивание данных с некорректным форматом. Все исключения при возникновении фиксируются в журнале событий с указанием уровня серьезности и контекста ошибки, а пользователю передаются информативные сообщения через интерфейс. Такой подход позволяет оперативно локализовать проблему, избежать прерывания всего пайплайна обработки и гарантировать, что на вход детекторам поступают только правильно распарсенные и агрегированные потоки.

§ 2.6. Реализованные детекторы

Для повышения устойчивости системы используется несколько независимых методов анализа, каждый из которых оценивает аномальность сетевого

потока с собственной точки зрения. Подобный подход позволяет снизить вероятность ложных срабатываний и повысить полноту обнаружения.

Перед передачей данных в модели, как и говорилось ранее, выполняется предварительная обработка признаков: стандартизация числовых значений, фильтрация пропусков, преобразование категориальных признаков и нормализация диапазонов.

§ 2.6.1. Детектор k -средних

Алгоритм KMeans используется для выделения групп сходных потоков. После масштабирования данных система разбивает множество объектов на кластеры, минимизируя внутрикластерное расстояние. В качестве меры аномальности используется расстояние объекта до центра ближайшего кластера.

Детектор в программе:

```
class KMeansDetector:
    def __init__(self, n_clusters: int = 3):
        self.n_clusters = n_clusters
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) < 2:
            return np.zeros(len(df), dtype=float)
        n_clusters = max(1, min(self.n_clusters, len(x)))
        if n_clusters == 1:
            return np.zeros(len(x), dtype=float)
        x_scaled = StandardScaler().fit_transform(x)
        model = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
        labels = model.fit_predict(x_scaled)
        centers = model.cluster_centers_
        distances = np.linalg.norm(x_scaled - centers[labels], axis=1)
        return normalize_scores(distances)
```

Этот класс использует KMeans для оценки «необычности» объектов (аномальности). На выходе возвращает оценку: чем выше значение, тем дальше точка от центра своей группы и тем больше она «выделяется» среди остальных точек.

Число кластеров в этом классе автоматически исправляется. Признаки анализируются, пересчитываются и адаптируются перед подачей в класс.

§ 2.6.2. Лес изоляции

Лес изоляции основан на случайной изоляции объектов. Алгоритм строит ансамбль деревьев, в которых данные разделяются случайными порогами. Аномальные объекты, как правило, изолируются быстрее нормальных.

Детектор в программе:

```
class IsolationForestDetector:
    def __init__(self, contamination: float = 0.1):
        self.contamination = contamination
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) < 2:
            return np.zeros(len(df), dtype=float)
        x_scaled = StandardScaler().fit_transform(x)
        model = IsolationForest(
            n_estimators=100,
            contamination=self.contamination,
            random_state=42,
        )
        model.fit(x_scaled)
        scores = -model.decision_function(x_scaled)
        return normalize_scores(scores)
```

В отличие от KMeans, он специально спроектирован для поиска выбросов: аномальные точки «изолируются» случайными разбиениями за меньшее число шагов, чем обычные.

§ 2.6.3. SVM

Метод опорных векторов SVM формирует границу нормального поведения в пространстве признаков. Все точки, которые находятся за пределами этой области, рассматриваются как потенциальные аномалии. Оптимизационная задача включает минимизацию нормы весов и штрафов за нарушения границы.

Детектор в программе:

```
class SVMDetector:
    def __init__(self, nu: float = 0.1):
        self.nu = nu
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) < 2:
            return np.zeros(len(df), dtype=float)
        x_scaled = StandardScaler().fit_transform(x)
        model = OneClassSVM(kernel="rbf", gamma="scale", nu=self.nu)
        model.fit(x_scaled)
```

```
scores = -model.decision_function(x_scaled)
return normalize_scores(scores)
```

Этот детектор явно учится отделять «нормальные» данные от всего остального: строит гибкую границу вокруг основной массы точек, а объекты за пределами этой границы считает выбросами.

§ 2.6.4. Статистический детектор

Статистический детектор вычисляет отклонение признаков потока от средних значений по набору данных. В его основе лежит z -score. Если отдельный признак демонстрирует значительное отклонение, общий уровень подозрительности потока возрастает.

Детектор в программе:

```
class StatisticalDetector:
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) == 0:
            return np.array([], dtype=float)
        if x.shape[1] == 0:
            return np.zeros(len(df), dtype=float)
        mean = x.mean(axis=0)
        std = x.std(axis=0)
        std[std == 0] = 1e-9 \
        z_scores = np.abs((x - mean) / std)
        max_z = z_scores.max(axis=1)
        return normalize_scores(max_z)
```

Он не обучается, а сразу оценивает, насколько каждая точка отклоняется от среднего по каждому признаку.

§ 2.6.5. Правилковой детектор

Правилковой детектор основан на эвристиках предметной области. Он анализирует используемые порты, тип протокола, соотношение bytes/packets и признаки слишком коротких или слишком объемных соединений. В нем заранее заданы «плохие» значения и, сравнивая с ними, детектор срабатывает на аномальность каждый раз при очередном совпадении.

§ 2.6.6. Модель машинного обучения

В качестве основной умной модели был выбран алгоритм случайного леса. Выбор данного алгоритма обусловлен устойчивостью к шуму, хорошей работой

на табличных данных, интерпретируемостью и сравнительно низким риском переобучения. Также сравнив с другими алгоритмами машинного обучения можно сделать вывод, что случайный лес требует меньшего числа гиперпараметров и проще интерпретируется. Данный алгоритм не ищет аномалии с нуля, а оценивает объекты с помощью уже готовой модели, обученной на размеченных данных.

Реализация:

```
class MLDetector:
    def __init__(self):
        self.model = None
        if os.path.exists(MODEL_PATH):
            try:
                self.model = joblib.load(MODEL_PATH)
            except Exception:
                self.model = None
    def predict(self, df) -> np.ndarray:
        X = get_numeric_matrix(df)
        if len(X) == 0:
            return np.array([], dtype=float)
        if self.model is None or X.shape[1] == 0:
            return np.zeros(len(X), dtype=float)
        try:
            if hasattr(self.model, "predict_proba"):
                probs = self.model.predict_proba(X)
                if probs.shape[1] >= 2:
                    return normalize_scores(probs[:, 1])
                    return normalize_scores(probs[:, 0])
                scores = self.model.decision_function(X)
                return normalize_scores(-scores)
            except Exception:
                return np.zeros(len(X), dtype=float)
```

§ 2.6.7. Сравнительный анализ работы детекторов на тестовых данных

Чтобы понять, насколько хорошо работает система, мы протестировали ее на специальном наборе данных. В нем смешаны два типа трафика: обычный, легитимный (как в реальной сети) и искусственно созданные аномалии. Конкретно мы включили в тесты:

- нормальные пользовательские сессии;
- сканирование портов (port scanning);
- попытки подбора паролей (brute force);

- активность, похожую на DDoS-атаку;
- аномально большую передачу данных (утечка данных).

Перед запуском экспериментов мы разделили этот датасет на две части: 80 процентов данных пошли на обучение модели (чтобы она «научилась» различать хорошее и плохое), а оставшиеся 20 процентов мы оставили для честной проверки — чтобы оценить, как модель справится с данными, которых она раньше не видела.

Для оценки качества мы использовали четыре основные метрики:

- точность (Pr) — показывает, какой процент из тех соединений, которые система назвала «подозрительными», действительно были атаками. Высокая точность означает мало ложных тревог;
- полнота (Rec) — отвечает на вопрос: «Какую долю реальных атак мы смогли обнаружить?» Высокая Полнота значит, что система мало что пропускает;
- F1-score — это нечто среднее гармоническое между точностью и полнотой. Удобная метрика, когда нужно найти баланс: чтобы и не пропускать угрозы, и не заваливать аналитику ложными срабатываниями;
- ROC-AUC — показывает, насколько хорошо модель в принципе умеет отделять «норму» от «аномалии» при разных порогах чувствительности. Значение ближе к 1.0 — отлично, 0.5 — как угадывание подбрасыванием монетки.

Формулы расчета берем следующие:

- $Pr = TP / (TP + FP)$;
- $Rec = TP / (TP + FN)$;
- $F1 = 2 \cdot (Pr \cdot Rec) / (Pr + Rec)$;

где TP (True Positive) — верно обнаруженные атаки, FP (False Positive) — ложные тревоги, FN (False Negative) — пропущенные атаки.

Тестирование показало, что отдельные алгоритмы демонстрируют различную чувствительность к типам аномалий. Наиболее высокое качество

Таблица 2.2. Сравнение методов по метрикам эффективности.

Метод	Pr	Rec	F1-score	ROC-AUC
<i>k</i> -средних	0.79	0.72	0.75	0.77
Лес изоляции	0.85	0.81	0.83	0.86
SVM	0.82	0.78	0.80	0.84
Статистический детектор	0.71	0.65	0.68	0.70
Случайный лес	0.89	0.86	0.87	0.91

показала ансамблевая модель случайного леса, но использование нескольких независимых детекторов позволяет уменьшить вероятность пропуска аномалий. Результаты получены в таблице 2.2.

Дополнительно был проведен анализ ложных срабатываний. Основная часть пропущенных атак наблюдалась для нестандартных, но законных соединений с редкими портами и атипичными объемами трафика. Использование нечеткой агрегации позволило уменьшить влияние одиночных ошибочных оценок.

Отдельно оценивалось влияние порога anomaly score на качество обнаружения. Наиболее сбалансированные результаты были получены при пороге 0.65 – 0.7. В ходе тестирования также анализировалось время обработки данных. Среднее время анализа файла среднего размера составляло несколько секунд.

§ 2.6.8. Анализ производительности системы

Производительность системы зависит от объема входных данных, числа потоков и используемых алгоритмов. Наиболее дорогостоящими в ресурсах компонентами являются SVM и анализ и обработка (парсинг) больших PCAP-файлов. Парсинг происходит следующим образом:

- 1) предварительная агрегация пакетов;
- 2) минимизация повторных вычислений;
- 3) кэширование промежуточных результатов;
- 4) пакетная обработка потоков;
- 5) ограничение числа признаков.

При увеличении объема данных система сохраняет линейный характер роста времени обработки для большинства компонентов, за исключением отдельных алгоритмов с более высокой вычислительной сложностью.

§ 2.7. Модуль нечеткой логики

После получения частных оценок от всех детекторов система переходит к этапу их объединения. Для этого используется упрощенная нечеткая логика. Выбор упрощенного подхода вместо полноценной системы Сугено обусловлен меньшей вычислительной сложностью, высокой скоростью работы, простотой интерпретации и достаточностью для задачи агрегирования оценочных показателей.

Каждый детектор возвращает числовую оценку в диапазоне $[0,1]$. Далее выполняется фаззификация: LOW — $x < 0.3$, MEDIUM — $0.3 \leq x < 0.7$, HIGH — $x \geq 0.7$ (здесь: LOW = низкий порог, MEDIUM = средний порог, HIGH = высокий порог).

Модуль фаззификации — перевод числовой оценки в лингвистическую переменную:

```
def _fuzzify(x: float) -> str:
    if x < 0.3:
        return "LOW"
    elif x < 0.7:
        return "MEDIUM"
    else:
        return "HIGH"
```

После фаззификации применяется набор правил ЕСЛИ-ТО (IF—THEN). Примеры правил: IF большинство оценок HIGH THEN итоговая аномальность HIGH; IF есть несколько HIGH и несколько MEDIUM THEN итоговая аномальность MEDIUM-HIGH; IF большинство оценок LOW THEN поток считается нормальным.

```
def _apply_rules(labels: list) -> float:
    high = sum(v == "HIGH" for v in labels)
    medium = sum(v == "MEDIUM" for v in labels)
    n = len(labels)
    if n == 0:
        return 0.0
    ratio_high = high / n
    ratio_medium = medium / n
    if ratio_high >= 0.6:
        return 0.9
    elif ratio_high >= 0.4:
```

```

        return 0.75
    elif ratio_high >= 0.3 and ratio_medium >= 0.3:
        return 0.6
    elif ratio_medium >= 0.6:
        return 0.45
    elif ratio_medium >= 0.2:
        return 0.3
    else:
        return 0.1

```

Пример расчета: метод k -средних = 0.82, Isolation Forest = 0.91, SVM = 0.76, Статистический = 0.44, Правильной метод = 0.88. После фаззификации получаем HIGH, HIGH, HIGH, MEDIUM, HIGH. Поскольку число HIGH ≥ 4 , итоговая оценка аномальности = 0.9.

Далее у нас идет сбор результатов детекторов, вызов этапов, возврат финальной оценки:

```

def fuzzy_score(
    kmeans_score: float,
    iforest_score: float,
    svm_score: float,
    stat_score: float,
    ml_score: float,
    bad_packet_score: float | None = None,
) -> float:
    labels = [
        _fuzzify(kmeans_score),
        _fuzzify(iforest_score),
        _fuzzify(svm_score),
        _fuzzify(stat_score),
        _fuzzify(ml_score),
    ]
    if bad_packet_score is not None:
        labels.append(_fuzzify(bad_packet_score))
    return _apply_rules(labels)

```

§ 2.8. Пользовательский интерфейс

Для обеспечения взаимодействия пользователя с системой обнаружения аномалий был разработан графический интерфейс на основе фреймворка Streamlit. Данный интерфейс выполняет роль прикладного уровня взаимодействия между пользователем и модулями анализа сетевого трафика. Через веб-интерфейс пользователь может загружать наборы данных, запускать процесс анализа, просматривать результаты обнаружения аномалий и визуализировать статистику работы алгоритмов.

Интерфейс реализован в виде одностраничного веб-приложения, взаимодействующего с внутренними модулями системы на языке Python. Архитектура взаимодействия построена по принципу передачи структурированных данных между компонентами обработки, визуализации и хранения результатов.

Были выполнены следующие пункты:

- 1) добавлена загрузка CSV и PCAP-файлов;
- 2) настройка порога аномальности;
- 3) отображение статистики;
- 4) визуализация результатов в виде графиков и диаграмм;
- 5) просмотр списка аномальных потоков.

Пользователь может изменять порог оценки аномальности, что позволяет управлять чувствительностью системы и балансировать между количеством ложных срабатываний и полнотой обнаружения.

§ 2.9. Преимущества разработанной системы

Разработанная система обладает рядом достоинств: использование нескольких независимых механизмов анализа, возможность работы как с CSV, так и с PCAP-файлами, наличие пользовательского интерфейса, интерпретируемость частных оценок и возможность дальнейшего расширения. Такой ансамблевый подход позволяет компенсировать слабые стороны отдельных алгоритмов, снижая количество ложных срабатываний без потери чувствительности к неизвестным угрозам. Кроме того, прозрачность логики принятия решений и удобный веб-интерфейс делают систему практичным инструментом для аналитиков, которым важно не просто получать метку «аномалия», но и понимать причины срабатывания детекторов.

Глава 3

Программная реализация

Эта глава сосредоточена на проверке работы описанной ранее программы. Здесь будет рассмотрено, как соотносятся этапы работы алгоритма с функциями и структурами программы, а также представлены скриншоты. Полный код разработанных в рамках этой работы программ представлен в приложениях.

Запускаем созданное приложение. Перед нами оказывается веб-интерфейс с понятным функционалом (рис. 3.1). Он обеспечивает взаимодействие с модулем анализа трафика, так чтобы простой пользователь мог легко в нем разобраться. В нем есть возможность настраивать порог аномальности, перенести с помощью функции drag and drop файл/датасет или загрузить этот же самый файл. Также есть возможность использовать тестовый файл.

Веб-интерфейс системы обеспечивает интуитивное взаимодействие с модулем анализа трафика. Интерфейс состоит из двух основных областей. Панель настроек (слева) содержит:

- 1) регулятор порога аномальности (0.0 — 1.0);
- 2) список активных детекторов (KMeans, Isolation Forest, One-Class SVM, статистический, Random Forest);
- 3) краткую справку по формату входных данных.

Рабочая область предоставляет:

- 1) загрузку файлов (.csv, .pcap, .pcapng) до 200 МБ;
- 2) опцию использования примера для демонстрации;
- 3) таблицу с примером структуры потока (IP-адреса, порты, протокол, объём данных).

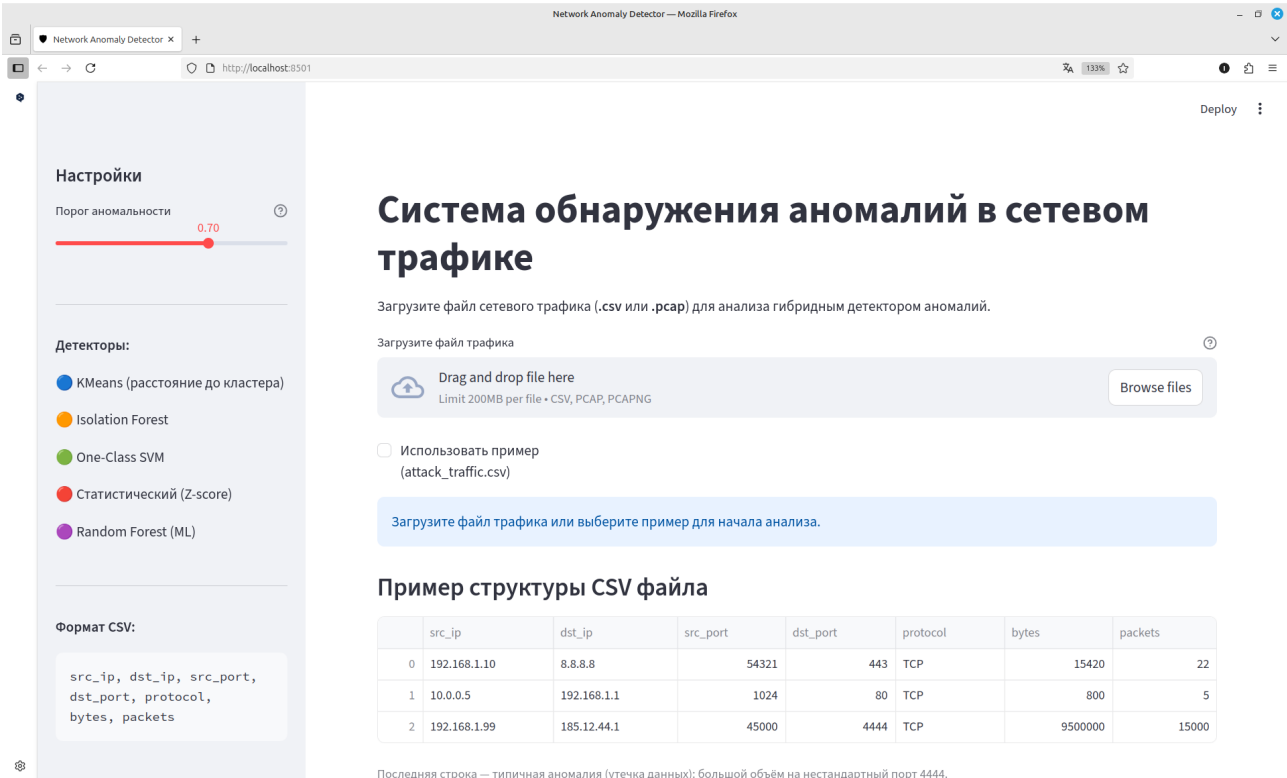


Рис. 3.1. Главный экран после запуска приложения.

Загружаем в качестве примера ранее подготовленный маленький набор данных, состоящий из 26 потоков (рис. 3.2). После загрузки у нас незамедлительно отрабатывают наши алгоритмы и методы. Посчитано количество потоков, количество аномалий (и их процент на фоне всех пакетов), количество нормальных пакетов и средняя оценка аномальности.

Всего потоков	Аномалий	Нормальных	Средний скор
26	3	23	0.319
	↑ 11.5%		

Рис. 3.2. Загруженные потоки.

Ниже программа предлагает ознакомиться с четырьмя вкладками (рис. 3.3).



Рис. 3.3. Вкладки для ознакомления с оценками методов программой.

На рисунке 3.4 показаны оценки аномальности потоков. На графиках можно отчетливо увидеть порог аномальности, количество аномальных потоков и какие потоки были аномальные.

Распределение гибридного сора аномальности



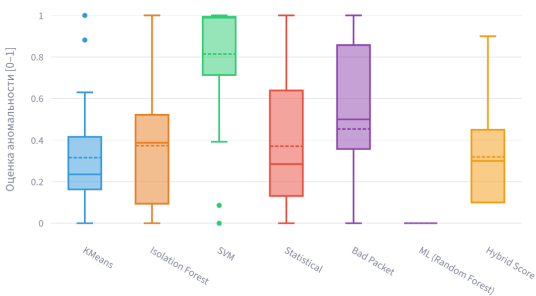
Оценка аномальности по потокам



Рис. 3.4. Оценки аномальности.

На рисунке 3.5 представлено сравнение методов обнаружения аномалий. Визуализация включает блочные диаграммы распределения, таблицу интегрального рейтинга и тепловую карту корреляций. Обращает внимание значительный разброс оценок. Метод SVM демонстрирует самые высокие средние значения и процент аномалий (более 80 %), что говорит о его высокой чувствительности. Random Forest и KMeans показали более консервативные результаты. Подобный разброс подтверждает правильность выбора гибридной архитектуры. Если бы система опиралась на один алгоритм, мы получили бы либо избыток ложных срабатываний (например с SVM), либо риск пропуска аномалий. Стоит отметить, что разброс отчасти обусловлен объемом тестовой выборки. На небольших данных алгоритмы могут вести себя по-разному. Это легко объяснить: с увеличением объема данных модели ведут себя лучше, увереннее. Именно интегральная оценка позволяет объединить эти очень разные мнения детекторов и получить объективный результат.

Сравнение оценок всех методов



Рейтинг методов (интегральная оценка)

	Метод	Mean	Std	Anomaly %	Score
2	SVM	0.814	0.278	80.8	0.594
4	Bad Packet	0.453	0.321	26.9	0.243
3	Statistical	0.371	0.301	15.4	0.171
1	Isolation Forest	0.373	0.294	11.5	0.162
6	Hybrid Score	0.319	0.257	11.5	0.143
0	KMeans	0.316	0.243	7.7	0.132
5	ML (Random Forest)	0	0	0	0

Корреляция между методами

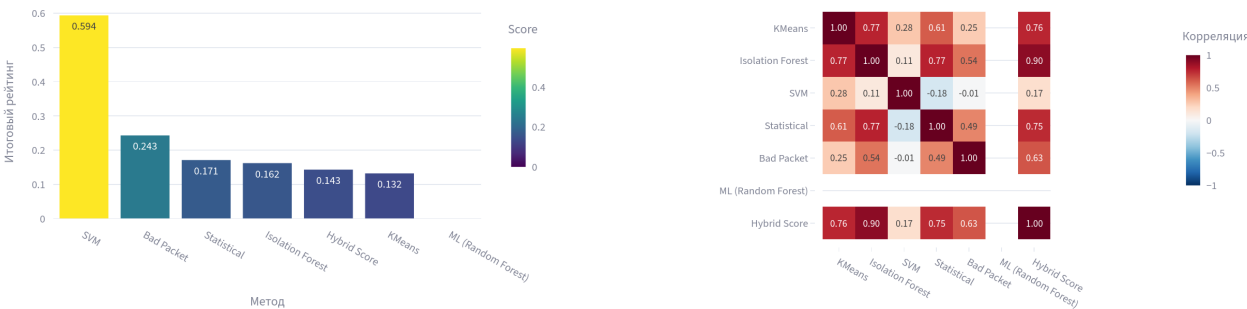
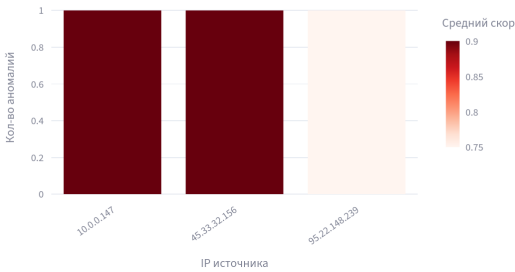


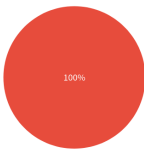
Рис. 3.5. Сравнение методов.

На рисунке 3.6 показана детализация по аномальному трафику: какие IP-адреса, протоколы чаще всего замечены как «аномальные», и как на это влияет количество отправленных байт и пакетов.

Топ IP-адресов источников аномального трафика



По протоколам (аномалии)



Bytes vs Packets (scatter)

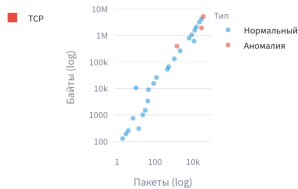


Рис. 3.6. Трафик.

На рисунке 3.7 (заключительной вкладке) мы можем посмотреть детализацию аномальных потоков, а также скачать список аномальных потоков (с их характеристиками) в виде файла csv.

[illegible]

	src_ip	dst_ip	src_port	dst_port	protocol	bytes	packets	kmeans_score	iforest_score	svm_score	stat_score	bad_packet_score	ml_score	anomaly_score	is_anomaly
0	192.168.1.114	192.168.1.237	12431	25	TCP	3340	42	0.1785	0.0027	0.9936	0.1673	0.3571	0	0.1	☐
1	192.168.0.120	10.0.0.231	16270	21	UDP	10492	10	0.1425	0.0372	0.9222	0.1148	0.3571	0	0.1	☐
2	192.168.0.77	216.125.193.198	18094	80	TCP	1485	31	0.2325	0.0941	0.8223	0.2856	0	0	0.1	☐
3	10.0.0.214	172.16.0.204	30844	3389	TCP	747	7	0.1627	0.2978	0.6419	0	0.3571	0	0.3	☐
4	10.0.0.147	102.19.140.93	18636	9999	TCP	390251	1425	1	0.9797	0.9892	0.8613	0.8571	0	0.9	☒
5	14.156.16.139	172.16.0.137	51027	443	TCP	611984	11370	0.4155	0.5218	0.0859	0.7767	0.5	0	0.3	☐
6	10.0.0.160	192.168.0.78	32928	22	TCP	1009	23	0.0991	0.0529	0.7256	0.091	0.3571	0	0.1	☐
7	95.22.148.239	192.168.1.42	10858	53	TCP	1906730	28038	0.8817	1	0.9909	0.6439	0.5	0	0.75	☒
8	192.168.1.216	159.52.14.58	4158	80	TCP	305	14	0.3511	0.3683	1	0.2856	0	0	0.3	☐

Заключение

В ходе выполнения работы были рассмотрены теоретические основы анализа сетевого трафика, методы обнаружения аномалий и принципы нечеткой логики. Показано, что для задач сетевой безопасности особенно эффективен гибридный подход, в котором сочетаются статистический анализ, алгоритмы машинного обучения, экспертные правила и нечеткое агрегирование.

На практическом уровне была спроектирована и реализована программная система, позволяющая анализировать сетевые потоки, вычислять частные оценки аномальности и формировать итоговое решение. Разработанное решение может служить основой для дальнейшего совершенствования и расширения функциональности в области обнаружения сетевых инцидентов.

Список литературы

- [1] *Bejtlich, R.* The Tao of Network Security Monitoring: Beyond Intrusion Detection / R. Bejtlich. — Indianapolis : Wiley, 2013. — 368 p. — (Digital Fortress).
- [2] Extended Isolation Forest for Intrusion Detection in Zeek Data / F. Moomtaheen, S. S. Bagui, S. C. Bagui, [et al.] // Information. — 2024. — URL: <https://www.mdpi.com/2078-2489/15/7/404> (visited on 03/12/2026).
- [3] *Al-Safaar, D. W.* A survey of network intrusion detection systems based on deep learning approaches / D. W. Al-Safaar, W. L. Al-Yaseen // Scientific and Technical Journal of Information Technologies, Mechanics and Optics. — 2023. — Vol. 23, no. 2. — P. 352–363.
- [4] *Spiekermann, D.* Unsupervised packet-based anomaly detection in virtual networks / D. Spiekermann, J. Keller // Computer Networks. — 2021. — URL: <https://www.sciencedirect.com/science/article/pii/S1389128621001286> (visited on 03/13/2026).
- [5] TCP/IP Tutorial and Technical Overview / L. Parziale, D. T. Britt, C. Davis, [et al.]. — 8th ed. — Poughkeepsie : IBM Redbooks, 2006. — 998 p. — (GG24-3376-07).
- [6] *Зайцев, О. В.* ROOTKITS, SPYWARE/ADWARE, KEYLOGGERS & BACKDOORS: обнаружение и защита / О. В. Зайцев. — СПб. : БХВ-Петербург, 2014. — 299 с. — ил.
- [7] *Карачанская, Е. В.* Метод выявления аномалий сетевого трафика, основанный на его самоподобной структуре / Е. В. Карачанская, Н. И. Соседова // Безопасность информационных технологий. — 2019. — Т. 26, № 1. — С. 98–110.
- [8] *Кугаевских, А. В.* Классические методы машинного обучения / А. В. Кугаевских, Д. И. Муромцев, О. В. Кирсанова. — СПб., 2022. — 53 с.

-
- [9] *Леоненков, А. В.* Нечеткое моделирование в среде MATLAB и fuzzyTECH / А. В. Леоненков. — СПб. : БХВ-Петербург, 2005. — 736 с.
- [10] *Олифер, В.* Компьютерные сети. Принципы, технологии, протоколы / В. Олифер, Н. Олифер. — Юбилейное издание. — СПб. : Питер, 2021. — 1008 с.
- [11] *Шелухин, О. И.* Сетевые аномалии. Обнаружение, локализация, прогнозирование / О. И. Шелухин. — Москва : Горячая линия – Телеком, 2020. — 448 с.
- [12] *Шелухин, О. И.* Обнаружение вторжений в компьютерные сети / О. И. Шелухин, Д. Ж. Сакалема, А. С. Филинова. — Москва : Горячая линия – Телеком, 2015. — 220 с.

Приложение А

Код программы

Архитектура программы

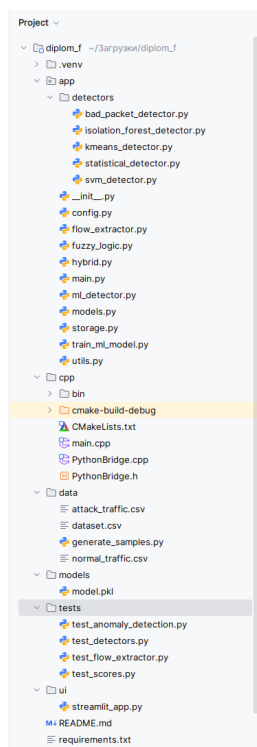


Рис. А.1. Архитектура программы.

Cmake

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(DiplomProject LANGUAGES CXX)
```

```

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Python3 COMPONENTS Interpreter Development REQUIRED)

add_executable(${PROJECT_NAME}
    main.cpp
    PythonBridge.cpp
    main.cpp
    PythonBridge.cpp
)

target_include_directories(${PROJECT_NAME} PRIVATE
    ${Python3_INCLUDE_DIRS}
    ${CMAKE_CURRENT_SOURCE_DIR}
)

target_link_libraries(${PROJECT_NAME} PRIVATE
    ${Python3_LIBRARIES}
)

target_compile_definitions(${PROJECT_NAME} PRIVATE
    PROJECT_SOURCE_DIR="${CMAKE_SOURCE_DIR}"
)

set_target_properties(${PROJECT_NAME} PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_SOURCE_DIR}/bin"
)

```

main.cpp

```

#include <cstdlib>
#include <string>
#ifdef PROJECT_SOURCE_DIR
#define PROJECT_SOURCE_DIR "."
#endif //ifndef PROJECT_SOURCE_DIR

int main() {
    std::string run_py_path = std::string(PROJECT_SOURCE_DIR) + "../run.py"
    ↪ ";

    std::string cmd = "python3 " + run_py_path;
    return system(cmd.c_str());
}

```

PythonBridge.cpp


```

#include "PythonBridge.h"
#include <cstdlib>
#include <iostream>
#include <string>
int PythonBridge::runScript(const std::string& script, const std::string&
↪ args) {
    std::string cmd = "python3 " + script + " " + args;
    std::cout << "[C++] start Python: " << cmd << std::endl;
    int result = system(cmd.c_str());
    if (result != 0) {
        std::cerr << "[C++] Error with compile" << script << std::endl;
    }
    return result;
}

```

PythonBridge.h

```

#pragma once
#include <string>
class PythonBridge {
public:
    static int runScript(const std::string& script, const std::string& args
↪ = "");
};

```

Детекторы

bad_packet_detector.py

```

import numpy as np
from app.utils import normalize_scores, get_numeric_matrix
class BadPacketDetector:
    fishy_ports = {21, 22, 23, 25, 110, 135, 139, 143, 445, 3389, 4444,
↪ 5555, 8080, 9999, 12345, 31337}
    def predict(self, df) -> np.ndarray:
        if df is None or len(df) == 0:
            return np.array([], dtype=float)
        x = get_numeric_matrix(df)
        bytes_idx = None
        packets_idx = None
        cols = list(df.select_dtypes(include=[np.number]).columns)
        if "bytes" in cols:
            bytes_idx = cols.index("bytes")
        elif "byte_count" in cols:
            bytes_idx = cols.index("byte_count")
        if "packets" in cols:

```

```

        packets_idx = cols.index("packets")
    elif "packet_count" in cols:
        packets_idx = cols.index("packet_count")
    scores = np.zeros(len(df), dtype=float)
    for i, (_, row) in enumerate(df.iterrows()):
        score = 0.0
        dst_port = int(row.get("dst_port", 0) or 0)
        src_port = int(row.get("src_port", 0) or 0)
        protocol = str(row.get("protocol", "")).upper()
        if dst_port in self.fishy_ports:
            score += 0.25
        if src_port in self.fishy_ports:
            score += 0.10
        if protocol not in {"TCP", "UDP", "ICMP", "OTHER"}:
            score += 0.10
        if bytes_idx is not None and packets_idx is not None:
            bytes_val = float(x[i, bytes_idx])
            packets_val = float(x[i, packets_idx])
            if packets_val <= 3 and bytes_val <= 200:
                score += 0.35
            if packets_val >= 1000 or bytes_val >= 1_000_000:
                score += 0.35
            if packets_val > 0 and bytes_val / packets_val > 50_000:
                score += 0.20
        scores[i] = min(score, 1.0)
    return normalize_scores(scores)

```

isolation_forest_detector.py

```

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import numpy as np
from app.utils import normalize_scores, get_numeric_matrix
class IsolationForestDetector:
    def __init__(self, contamination: float = 0.1):
        self.contamination = contamination
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) < 2:
            return np.zeros(len(df), dtype=float)
        x_scaled = StandardScaler().fit_transform(x)
        model = IsolationForest(
            n_estimators=100,
            contamination=self.contamination,
            random_state=42,
        )
        model.fit(x_scaled)
        scores = -model.decision_function(x_scaled)
        return normalize_scores(scores)

```

kmeans_detector.py

```

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import numpy as np
from app.utils import normalize_scores, get_numeric_matrix
class KMeansDetector:
    def __init__(self, n_clusters: int = 3):
        self.n_clusters = n_clusters
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) < 2:
            return np.zeros(len(df), dtype=float)
        n_clusters = max(1, min(self.n_clusters, len(x)))
        if n_clusters == 1:
            return np.zeros(len(x), dtype=float)
        x_scaled = StandardScaler().fit_transform(x)
        model = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
        labels = model.fit_predict(x_scaled)
        centers = model.cluster_centers_
        distances = np.linalg.norm(x_scaled - centers[labels], axis=1)
        return normalize_scores(distances)

```

statistical_detector.py

```

import numpy as np
from app.utils import normalize_scores, get_numeric_matrix
class StatisticalDetector:
    def predict(self, df) -> np.ndarray:
        x = get_numeric_matrix(df)
        if x.size == 0 or len(x) == 0:
            return np.array([], dtype=float)
        if x.shape[1] == 0:
            return np.zeros(len(df), dtype=float)
        mean = x.mean(axis=0)
        std = x.std(axis=0)
        std[std == 0] = 1e-9
        z_scores = np.abs((x - mean) / std)
        max_z = z_scores.max(axis=1)
        return normalize_scores(max_z)

```

svm_detector.py

```

class SVMDetector:
    def __init__(self, nu: float = 0.1):
        self.nu = nu

```

```

def predict(self, df) -> np.ndarray:
    x = get_numeric_matrix(df)
    if x.size == 0 or len(x) < 2:
        return np.zeros(len(df), dtype=float)
    x_scaled = StandardScaler().fit_transform(x)
    model = OneClassSVM(kernel="rbf", gamma="scale", nu=self.nu)
    model.fit(x_scaled)
    scores = -model.decision_function(x_scaled)
    return normalize_scores(scores)

```

hybryd.py

```

import pandas as pd
from app.detectors.kmeans_detector import KMeansDetector
from app.detectors.isolation_forest_detector import IsolationForestDetector
from app.detectors.svm_detector import SVMDetector
from app.detectors.statistical_detector import StatisticalDetector
from app.detectors.bad_packet_detector import BadPacketDetector
from app.ml_detector import MLDetector
from app.fuzzy_logic import fuzzy_score
from app.config import DEFAULT_THRESHOLD
class HybridDetector:
    def __init__(self):
        self.kmeans = KMeansDetector()
        self.iforest = IsolationForestDetector()
        self.svm = SVMDetector()
        self.stat = StatisticalDetector()
        self.bad_packets = BadPacketDetector()
        self.ml = MLDetector()
    def predict(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()
        if df.empty:
            for col in [
                "kmeans_score", "iforest_score", "svm_score",
                "stat_score", "bad_packet_score", "ml_score",
                "anomaly_score", "is_anomaly",
            ]:
                df[col] = []
            return df
        df["kmeans_score"] = self.kmeans.predict(df)
        df["iforest_score"] = self.iforest.predict(df)
        df["svm_score"] = self.svm.predict(df)
        df["stat_score"] = self.stat.predict(df)
        df["bad_packet_score"] = self.bad_packets.predict(df)
        df["ml_score"] = self.ml.predict(df)
        df["anomaly_score"] = df.apply(
            lambda row: fuzzy_score(
                row["kmeans_score"],
                row["iforest_score"],
                row["svm_score"],

```

```

        row["stat_score"],
        row["ml_score"],
        row["bad_packet_score"],
    ),
    axis=1,
)
df["is_anomaly"] = df["anomaly_score"] >= DEFAULT_THRESHOLD
return df

```

ml_detector.py

```

import os
import numpy as np
import joblib
from app.config import MODEL_PATH
from app.utils import normalize_scores, get_numeric_matrix
class MLDetector:
    def __init__(self):
        self.model = None
        if os.path.exists(MODEL_PATH):
            try:
                self.model = joblib.load(MODEL_PATH)
            except Exception:
                self.model = None
    def predict(self, df) -> np.ndarray:
        X = get_numeric_matrix(df)
        if len(X) == 0:
            return np.array([], dtype=float)
        if self.model is None or X.shape[1] == 0:
            return np.zeros(len(X), dtype=float)
        try:
            if hasattr(self.model, "predict_proba"):
                probs = self.model.predict_proba(X)
                if probs.shape[1] >= 2:
                    return normalize_scores(probs[:, 1])
                    return normalize_scores(probs[:, 0])
                scores = self.model.decision_function(X)
                return normalize_scores(-scores)
            except Exception:
                return np.zeros(len(X), dtype=float)

```

config.py

```

DB_URL = "sqlite:///results.db"
MODEL_PATH = "models/model.pkl"
DEFAULT_THRESHOLD = 0.7

```

models.py

```
from sqlalchemy import Column, Integer, Float, String
from app.storage import Base
class Alert(Base):
    __tablename__ = "alerts"
    id = Column(Integer, primary_key=True)
    src_ip = Column(String)
    dst_ip = Column(String)
    score = Column(Float)
```

Парсинг

flow_extractor.py

```
import os
import pandas as pd
def extract_flows(path: str) -> pd.DataFrame:
    ext = os.path.splitext(path)[1].lower()
    if ext in (".pcap", ".pcapng"):
        return _extract_from_pcap(path)
    elif ext == ".csv":
        return _extract_from_csv(path)
    else:
        raise ValueError(fbad format file: {ext})
def _extract_from_pcap(path: str) -> pd.DataFrame:
    try:
        from scapy.all import rdpcap, IP, TCP, UDP
    except ImportError:
        raise ImportError("pip install scapy")
    packets = rdpcap(path)
    flows: dict = {}
    for pkt in packets:
        if IP not in pkt:
            continue
        src = pkt[IP].src
        dst = pkt[IP].dst
        proto = "OTHER"
        sport = 0
        dport = 0
        if TCP in pkt:
            proto = "TCP"
            sport = pkt[TCP].sport
            dport = pkt[TCP].dport
        elif UDP in pkt:
            proto = "UDP"
            sport = pkt[UDP].sport
```

```

        dport = pkt[UDP].dport
        key = (src, dst, sport, dport, proto)
        if key not in flows:
            flows[key] = {"bytes": 0, "packets": 0}
        flows[key]["bytes"] += len(pkt)
        flows[key]["packets"] += 1
    rows = []
    for k, v in flows.items():
        rows.append({
            "src_ip": k[0],
            "dst_ip": k[1],
            "src_port": k[2],
            "dst_port": k[3],
            "protocol": k[4],
            "bytes": v["bytes"],
            "packets": v["packets"],
        })
    return pd.DataFrame(rows)

def _extract_from_csv(path: str) -> pd.DataFrame:
    required = {"src_ip", "dst_ip", "src_port", "dst_port", "protocol", "
    ↪ bytes", "packets"}
    df = pd.read_csv(path)
    missing = required - set(df.columns)
    if missing:
        raise ValueError(f"In csv miss column: {missing}")
    return df

```

utils.py

```

import logging
import numpy as np

def setup_logger():
    logger = logging.getLogger("IDS")
    logger.setLevel(logging.INFO)
    if not logger.handlers:
        ch = logging.StreamHandler()
        formatter = logging.Formatter("%(asctime)s | %(levelname)s | %(
        ↪ message)s")
        ch.setFormatter(formatter)
        logger.addHandler(ch)
    return logger

logger = setup_logger()

def get_numeric_matrix(df) -> np.ndarray:
    if df is None:
        return np.empty((0, 0))
    X = df.select_dtypes(include=[np.number]).to_numpy(dtype=float, copy=
    ↪ False)
    if X.ndim == 1:
        X = X.reshape(-1, 1)
    return X

```

```
def normalize_scores(scores: np.ndarray) -> np.ndarray:
    scores = np.asarray(scores, dtype=float)
    if scores.size == 0:
        return scores
    finite = scores[np.isfinite(scores)]
    if finite.size == 0:
        return np.zeros_like(scores, dtype=float)
    min_val = finite.min()
    max_val = finite.max()
    if max_val - min_val == 0:
        return np.zeros_like(scores, dtype=float)
    scores = np.nan_to_num(scores, nan=min_val, posinf=max_val, neginf=
↪ min_val)
    return (scores - min_val) / (max_val - min_val)
```

Нечеткая логика

fuzzy_logic.py

```
def _fuzzify(x: float) -> str:
    if x < 0.3:
        return "LOW"
    elif x < 0.7:
        return "MEDIUM"
    else:
        return "HIGH"

def _apply_rules(labels: list) -> float:
    high = sum(v == "HIGH" for v in labels)
    medium = sum(v == "MEDIUM" for v in labels)
    n = len(labels)
    if n == 0:
        return 0.0
    ratio_high = high / n
    ratio_medium = medium / n
    if ratio_high >= 0.6:
        return 0.9
    elif ratio_high >= 0.4:
        return 0.75
    elif ratio_high >= 0.3 and ratio_medium >= 0.3:
        return 0.6
    elif ratio_medium >= 0.6:
        return 0.45
    elif ratio_medium >= 0.2:
        return 0.3
    else:
        return 0.1

def fuzzy_score(
    kmeans_score: float,
```



```

        iforest_score: float,
        svm_score: float,
        stat_score: float,
        ml_score: float,
        bad_packet_score: float | None = None,
    ) -> float:
        labels = [
            _fuzzify(kmeans_score),
            _fuzzify(iforest_score),
            _fuzzify(svm_score),
            _fuzzify(stat_score),
            _fuzzify(ml_score),
        ]
        if bad_packet_score is not None:
            labels.append(_fuzzify(bad_packet_score))
        return _apply_rules(labels)

```

Создание потоков

generate_samples.py

```

import os
import random
import numpy as np
import pandas as pd
random.seed(42)
np.random.seed(42)
OUTPUT_DIR = os.path.dirname(__file__)
NORMAL_PORTS = [80, 443, 53, 8080, 22, 25, 110, 143, 21, 3389]
PROTOCOLS = ["TCP", "UDP", "OTHER"]
INTERNAL_NETS = ["192.168.1.", "192.168.0.", "10.0.0.", "172.16.0."]
def random_ip(internal=True):
    if internal:
        net = random.choice(INTERNAL_NETS)
        return net + str(random.randint(1, 254))
    else:
        return f"{random.randint(1,223)}.{random.randint(0,255)}.{random.
↵ randint(0,255)}.{random.randint(1,254)}"
def generate_normal_flow():
    proto = random.choices(PROTOCOLS, weights=[0.6, 0.35, 0.05])[0]
    return {
        "src_ip": random_ip(internal=True),
        "dst_ip": random_ip(internal=random.random() > 0.4),
        "src_port": random.randint(1024, 65535),
        "dst_port": random.choice(NORMAL_PORTS),
        "protocol": proto,
        "bytes": int(np.random.lognormal(mean=8, sigma=1.5)),
        "packets": int(np.random.lognormal(mean=3, sigma=1.0)),
    }

```

```

        "label": 0,
    }
def generate_portscan_flow():
    return {
        "src_ip": random_ip(internal=False),
        "dst_ip": random_ip(internal=True),
        "src_port": random.randint(1024, 65535),
        "dst_port": random.randint(1, 65535),
        "protocol": "TCP",
        "bytes": random.randint(40, 200),
        "packets": random.randint(1, 3),
        "label": 1,
    }
def generate_ddos_flow():
    victim_ip = random_ip(internal=True)
    attacker = random_ip(internal=False)
    return {
        "src_ip": attacker,
        "dst_ip": victim_ip,
        "src_port": random.randint(1024, 65535),
        "dst_port": random.choice([80, 443, 53]),
        "protocol": random.choice(["TCP", "UDP"]),
        "bytes": int(np.random.lognormal(mean=14, sigma=0.5)),
        "packets": int(np.random.lognormal(mean=10, sigma=0.5)),
        "label": 1,
    }
def generate_data_exfiltration_flow():
    return {
        "src_ip": random_ip(internal=True),
        "dst_ip": random_ip(internal=False),
        "src_port": random.randint(1024, 65535),
        "dst_port": random.choice([4444, 5555, 9999, 31337, 12345]),
        "protocol": "TCP",
        "bytes": int(np.random.lognormal(mean=13, sigma=1.0)),
        "packets": int(np.random.lognormal(mean=7, sigma=0.8)),
        "label": 1,
    }
def generate_bruteforce_flow():
    return {
        "src_ip": random_ip(internal=False),
        "dst_ip": random_ip(internal=True),
        "src_port": random.randint(1024, 65535),
        "dst_port": random.choice([22, 3389, 23, 21]),
        "protocol": "TCP",
        "bytes": random.randint(200, 2000),
        "packets": random.randint(5, 30),
        "label": 1,
    }
def generate_dataset(n_normal=800, n_anomaly=200):
    rows = []
    for _ in range(n_normal):
        rows.append(generate_normal_flow())

```

```

    attack_generators = [
        generate_portscan_flow,
        generate_ddos_flow,
        generate_data_exfiltration_flow,
        generate_bruteforce_flow,
    ]
    for i in range(n_anomaly):
        gen = attack_generators[i % len(attack_generators)]
        rows.append(gen())
    random.shuffle(rows)
    return pd.DataFrame(rows)
def generate_test_csv(n_normal=150, n_anomaly=50, filename="attack_traffic.
↪ csv"):
    rows = []
    for _ in range(n_normal):
        rows.append(generate_normal_flow())
    attack_gens = [generate_portscan_flow, generate_ddos_flow,
                    generate_data_exfiltration_flow, generate_bruteforce_flow
                    ↪ ]
    for i in range(n_anomaly):
        rows.append(attack_gens[i % len(attack_gens)]())
    random.shuffle(rows)
    df = pd.DataFrame(rows)
    path = os.path.join(OUTPUT_DIR, filename)
    df.to_csv(path, index=False)
    return path
if __name__ == "__main__":
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    print("Generation dataset for learning...")
    df_full = generate_dataset(n_normal=800, n_anomaly=200)
    path_full = os.path.join(OUTPUT_DIR, "dataset.csv")
    df_full.to_csv(path_full, index=False)
    print(f"    {path_full}    ({len(df_full)} strok, "
          f"{df_full['label'].sum()} anomaly)")
    print("generate normal trafic...")
    df_normal = pd.DataFrame([generate_normal_flow() for _ in range(200)])
    df_normal = df_normal.drop(columns=["label"])
    path_normal = os.path.join(OUTPUT_DIR, "normal_traffic.csv")
    df_normal.to_csv(path_normal, index=False)
    print(f"    {path_normal}    ({len(df_normal)} strok)")
    print("generation with attack...")
    path_attack = generate_test_csv(n_normal=150, n_anomaly=50)
    df_attack = pd.read_csv(path_attack).drop(columns=["label"], errors="
↪ ignore")
    df_attack.to_csv(path_attack, index=False)
    print(f"    {path_attack}    ({len(df_attack)} strok)")
    print("\nGreat! Learn model:")
    print("    python -m app.train_ml_model")

```

Интерфейс

streamlit_app.py

```

import sys
import os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from app.flow_extractor import extract_flows
from app.hybrid import HybridDetector
from app.config import DEFAULT_THRESHOLD
st.set_page_config(
    page_title="Network Anomaly Detector",
    page_icon="",
    layout="wide",
)
st.title("System of detecting anomaly")
st.markdown(
    "Browse file(**.csv** or **.pcap**) "
    "for analize with hybryd detector."
)
with st.sidebar:
    st.header("tools")
    threshold = st.slider(
        "Porog of anomaly",
        min_value=0.1,
        max_value=1.0,
        value=DEFAULT_THRESHOLD,
        step=0.05,
        help="Streams with score taller porogaare anomaly",
    )
    st.divider()
    st.markdown("**detectors:**")
    st.markdown("KMeans")
    st.markdown("Isolation Forest")
    st.markdown("One-Class SVM")
    st.markdown("(Z-score)")
    st.markdown("Random Forest (ML)")
    st.divider()
    st.markdown("**Format CSV:**")
    st.code(
        "src_ip, dst_ip, src_port,\n"
        "dst_port, protocol,\n"
        "bytes, packets",
        language="text",
    )
uploaded_file = st.file_uploader(

```

```

        "browse file of traffic",
        type=["csv", "pcap", "pcapng"],
        help="formats CSV and PCAP",
    )
    col_example, col_spacer = st.columns([2, 5])
    with col_example:
        use_example = st.checkbox("Example (attack_traffic.csv)")
    @st.cache_data(show_spinner=False)
    def load_and_analyze(file_bytes: bytes, filename: str, thresh: float) -> pd.
    ↪ DataFrame:
        import tempfile
        ext = os.path.splitext(filename)[1].lower()
        with tempfile.NamedTemporaryFile(delete=False, suffix=ext) as tmp:
            tmp.write(file_bytes)
            tmp_path = tmp.name
        try:
            df = extract_flows(tmp_path)
            detector = HybridDetector()
            result = detector.predict(df)
            result["is_anomaly"] = result["anomaly_score"] >= thresh
            return result
        finally:
            if os.path.exists(tmp_path):
                os.unlink(tmp_path)
    df = None
    if use_example:
        example_path = os.path.join(os.path.dirname(__file__), "..", "data", "
    ↪ attack_traffic.csv")
        if os.path.exists(example_path):
            with open(example_path, "rb") as f:
                file_bytes = f.read()
            with st.spinner("analyze example trafic."):
                df = load_and_analyze(file_bytes, "attack_traffic.csv",
    ↪ threshold)
        else:
            st.warning("File of example not found. Play: 'python data/
    ↪ generate_samples.py'")
    elif uploaded_file is not None:
        with st.spinner("Trafic analyze..."):
            df = load_and_analyze(uploaded_file.getvalue(), uploaded_file.name,
    ↪ threshold)
    if df is not None:
        anomalies = df[df["is_anomaly"]]
        normal = df[~df["is_anomaly"]]
        st.divider()
        m1, m2, m3, m4 = st.columns(4)
        m1.metric("Sum of streams", len(df))
        m2.metric("Anomaly", len(anomalies), delta=f"{len(anomalies)/len(df)
    ↪ *100:.1f}%")
        m3.metric("Norm", len(normal))
        m4.metric("Medium score", f"{df['anomaly_score'].mean():.3f}")
        st.divider()

```

```

tab1, tab2, tab3, tab4 = st.tabs(
    ["Score of anomaly", "Sravneny metodov", "Trafic", "Details"]
)
with tab1:
    st.subheader("Raspredileny score of anomalnost")
    fig = go.Figure()
    fig.add_trace(go.Histogram(
        x=normal["anomaly_score"],
        name="Norm",
        marker_color="\#2ecc71",
        opacity=0.75,
        nbinsx=40,
    ))
    fig.add_trace(go.Histogram(
        x=anomalies["anomaly_score"],
        name="Anomaly",
        marker_color="\#e74c3c",
        opacity=0.75,
        nbinsx=40,
    ))
    fig.add_vline(
        x=threshold,
        line_dash="dash",
        line_color="orange",
        annotation_text=f"Porog ({threshold})",
        annotation_position="top right",
    )
    fig.update_layout(
        barmode="overlay",
        xaxis_title="Score of anomaly",
        yaxis_title="sum of streams",
        legend=dict(orientation="h", yanchor="bottom", y=1.02),
        height=400,
    )
    st.plotly_chart(fig, use_container_width=True)
    st.subheader("Score of anomaly po streams")
    df_plot = df.reset_index(drop=True)
    colors = df_plot["is_anomaly"].map({True: "\#e74c3c", False: "\#3498
↵ db"})
    fig2 = go.Figure()
    fig2.add_trace(go.Scatter(
        x=df_plot.index,
        y=df_plot["anomaly_score"],
        mode="markers",
        marker=dict(color=colors, size=5, opacity=0.7),
        text=df_plot.apply(
            lambda r: f"{r['src_ip']}\rightarrow{r['dst_ip']} | {r['
↵ protocol']}" | score={r['anomaly_score']:.3f}",
            axis=1,
        ),
        hovertemplate="%{text}<extra></extra>",
    ))

```

```

fig2.add_hline(
    y=threshold,
    line_dash="dash",
    line_color="orange",
    annotation_text="porog",
)
fig2.update_layout(
    xaxis_title="Index of stream",
    yaxis_title="Score of anomaly",
    height=350,
)
st.plotly_chart(fig2, use_container_width=True)
with tab2:
    st.subheader("Sravneny score of all methods")
    score_cols = ["kmeans_score", "iforest_score", "svm_score", "
↪ stat_score", "bad_packet_score", "ml_score", "anomaly_score"]
    labels = ["KMeans", "Isolation Forest", "SVM", "Statistical", "Bad
↪ Packet", "ML (Random Forest)", "Hybrid Score"]
    colors_box = ["\#3498db", "\#e67e22", "\#2ecc71", "\#e74c3c", "\#8
↪ e44ad", "\#9b59b6", "\#f39c12"]
    fig3 = go.Figure()
    for col, label, color in zip(score_cols, labels, colors_box):
        fig3.add_trace(go.Box(
            y=df[col],
            name=label,
            marker_color=color,
            boxmean=True,
        ))
    fig3.update_layout(
        yaxis_title="Score of anomaly [0-1]",
        height=450,
        showlegend=False,
    )
    st.plotly_chart(fig3, use_container_width=True)
    st.subheader(" Rating of methods")
    ranking_data = []
    for col, label in zip(score_cols, labels):
        mean_score = df[col].mean()
        std_score = df[col].std()
        anomaly_ratio = (df[col] >= threshold).mean()
        score = (
            0.5 * mean_score +
            0.3 * anomaly_ratio -
            0.2 * std_score
        )
    ranking_data.append({
        "Method": label,
        "Mean": round(mean_score, 3),
        "Std": round(std_score, 3),
        "Anomaly %": round(anomaly_ratio * 100, 1),
        "Score": round(score, 3)
    })

```

```

ranking_df = pd.DataFrame(ranking_data).sort_values("Score",
↳ ascending=False)
st.dataframe(ranking_df, use_container_width=True)
fig_rank = px.bar(
    ranking_df,
    x="Method",
    y="Score",
    color="Score",
    color_continuous_scale="Viridis",
    text="Score",
    height=400,
)
fig_rank.update_layout(
    yaxis_title="Itog rating",
    xaxis_title="Method",
)
st.plotly_chart(fig_rank, use_container_width=True)
st.subheader("Korrelation")
corr = df[score_cols].corr()
fig4 = px.imshow(
    corr,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    labels=dict(color="Korrelation"),
    x=labels,
    y=labels,
)
fig4.update_layout(height=400)
st.plotly_chart(fig4, use_container_width=True)

with tab3:
    st.subheader("Top of anomaly IP")
    if not anomalies.empty:
        top_src = (
            anomalies.groupby("src_ip")
            .agg(count=("anomaly_score", "count"), avg_score=(
↳ anomaly_score", "mean"))
            .sort_values("count", ascending=False)
            .head(15)
            .reset_index()
        )
        fig5 = px.bar(
            top_src,
            x="src_ip",
            y="count",
            color="avg_score",
            color_continuous_scale="Reds",
            labels={"count": "Sum of anomalu", "src_ip": "IP source",
                "avg_score": "Middle score"},
            height=380,

```



```

    )
    fig5.update_layout(xaxis_tickangle=-35)
    st.plotly_chart(fig5, use_container_width=True)
else:
    st.info("anomaly not found.")
col_a, col_b = st.columns(2)
with col_a:
    st.subheader("on protocol")
    if not anomalies.empty:
        proto_counts = anomalies["protocol"].value_counts()
        fig6 = px.pie(
            values=proto_counts.values,
            names=proto_counts.index,
            color_discrete_sequence=["\#e74c3c", "\#e67e22", "\#9
↵ b59b6"],
            height=320,
        )
        st.plotly_chart(fig6, use_container_width=True)
with col_b:
    st.subheader("Bytes vs Packets (scatter)")
    df_scatter = df.copy()
    df_scatter["type"] = df_scatter["is_anomaly"].map(
        {True: "Anomaly", False: "Norm"}
    )
    fig7 = px.scatter(
        df_scatter,
        x="packets",
        y="bytes",
        color="type",
        color_discrete_map={"Anomaly": "\#e74c3c", "Norm": "\#3498db
↵ "},
        opacity=0.6,
        log_x=True,
        log_y=True,
        labels={"packets": "Packets (log)", "bytes": "Bytes (log)"},
        height=320,
    )
    st.plotly_chart(fig7, use_container_width=True)
with tab4:
    st.subheader("Anomaly streams")
    if not anomalies.empty:
        display_cols = [
            "src_ip", "dst_ip", "src_port", "dst_port", "protocol",
            "bytes", "packets", "anomaly_score",
            "kmeans_score", "iforest_score", "svm_score", "stat_score",
            ↵ "bad_packet_score", "ml_score",
        ]
    st.dataframe(
        anomalies[display_cols].sort_values("anomaly_score",
        ↵ ascending=False).reset_index(drop=True),
        use_container_width=True,
        height=400,
    )

```

```

    )
    csv_data = anomalies[display_cols].to_csv(index=False).encode("
↪ utf-8")
    st.download_button(
        "download anomaly (CSV)",
        data=csv_data,
        file_name="anomalies.csv",
        mime="text/csv",
    )
else:
    st.success("Anomalies not found. Trafic norm")
st.subheader("All streams")
st.dataframe(
    df.reset_index(drop=True),
    use_container_width=True,
    height=350,
)
else:
    st.info("Browse files")
    st.subheader("Example structure CSV file")
    example_df = pd.DataFrame({
        "src_ip":    ["192.168.1.10", "10.0.0.5",    "192.168.1.99"],
        "dst_ip":    ["8.8.8.8",      "192.168.1.1", "185.12.44.1"],
        "src_port":  [54321,          1024,         45000],
        "dst_port":  [443,            80,           4444],
        "protocol": ["TCP",           "TCP",         "TCP"],
        "bytes":     [15420,          800,           9500000],
        "packets":   [22,             5,            15000],
    })
    st.dataframe(example_df, use_container_width=True)
    st.caption("Last stroka- typical anomaly: bad port 4444.")

```